

Web Technology



Javascript – Datatypes, Variables, Type Conversion, Control

Structures, Arrays, Functions

Rourab Paul & Subin Sahayam
Department of Computer Science and
Engineering
Shiv Nadar University

Last Class Summary

- **Javascript Statistics**
- **Java vs Javascript**
- **Language**
- **Introduction**
- **Types of Programming**
- **Inline – Hello World - Inline**
- **External – Hello World – Inline**
- **External – Hello World - Alert**
- **Keywords**
- **Best Practices**

Last Class Syntax Summary

- `console.log(“”);`
- `alert(“”);`
- `document.write(“”);`

Introduction

Language

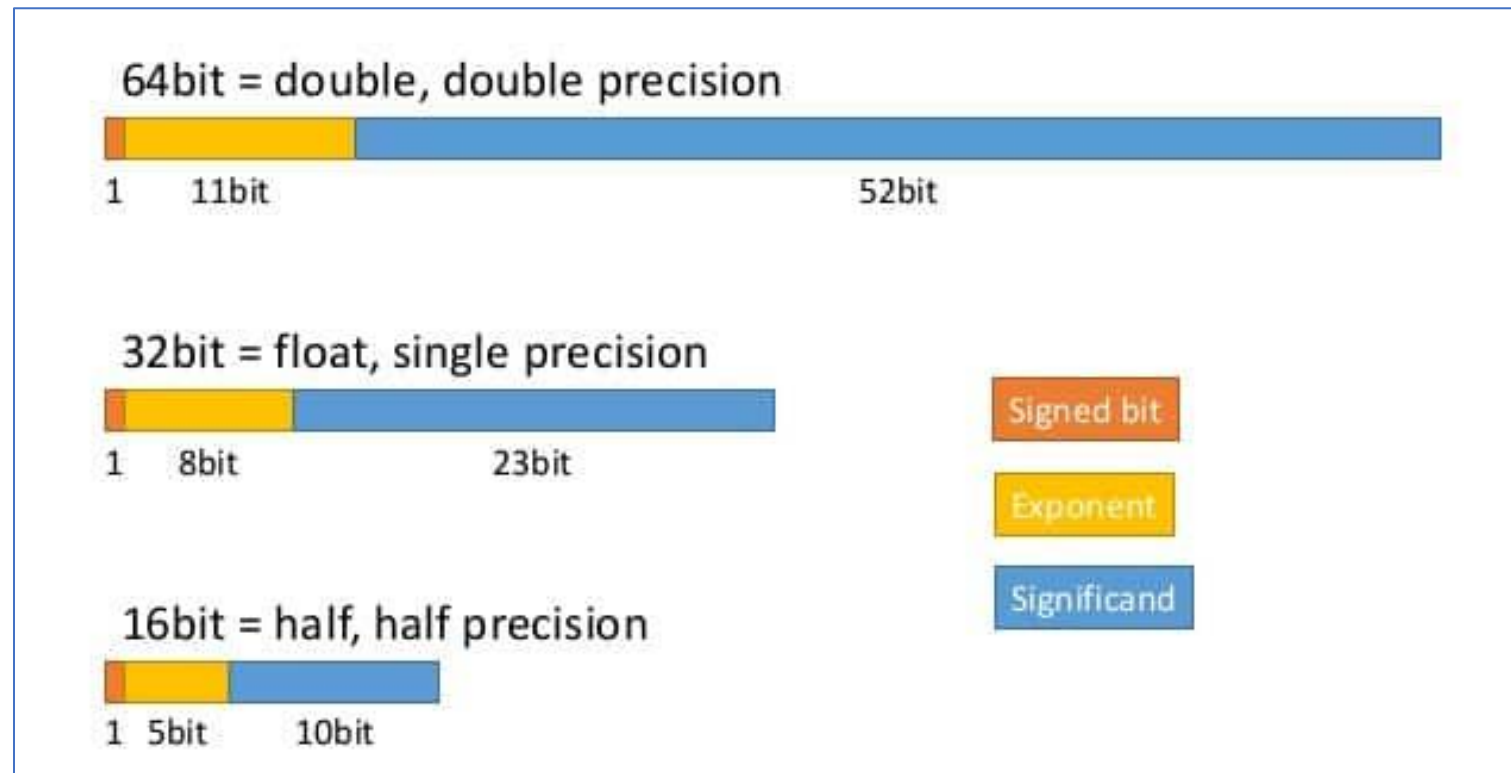
	English	Javascript	HTML
Alphabet	A-Z, a-z	A-Z, a-z	A-Z, a-z
Numbers	0-9	0-9, 0 and 1, 0-7, 0-F	0-9
Words	Words	Tokens	Nodes
Sentences	Grammar + Words = Sentences	Syntax + Tokens = Statements	Syntax + Nodes = Elements
Paragraph	Paragraph	Block	HTMLCollecti on
Chapter/Book	Chapter/Book	Program	Web Page
Library	Library	Library	Website

Data Types

- **integer and float – a single type - numbers**
 - **Double-precision – 64-bit**
 - **Eg = $\frac{1}{2}$**

Data Types

- integer and float – a single type - numbers
 - Double-precision – 64-bit
 - Eg = $\frac{1}{2}$



Data Types

- integer and float – a single type - numbers
 - Double-precision – 64-bit
 - Eg = $\frac{1}{2}$
 - C and Java – Output: 0
 - Javascript – Output: 0.5

```
let a = 10; // Integer
let b = 10.75; // Floating-point
console.log(typeof a); // number
console.log(typeof b); // number
```

Language	Code	Output
C	<code>int a=1, b=2; printf("%d", a/b);</code>	0
Java	<code>int a=1, b=2; System.out.println(a/b);</code>	0
JavaScript	<code>console.log(1/2);</code>	0.5

Data Types

- **integer and float – a single type - numbers**
 - **Double-precision – 64-bit**
 - **Eg = $\frac{1}{2}$**
 - **C and Java – Output: 0**
 - **Javascript – Output: 0.5**
- **Three Special Numbers**
 - **Infinity**
 - **-Infinity**
 - **NaN**

```
console.log(parseInt("Hello"));
```

Data Types

- **String**
 - Same as Java and C++
 - '+' concatenates
 - String with other/ same datatype values
 - Substring functions
- **Boolean**
 - true and false
- **BigInt**
 - Truly integer.
 - Create
 - **BigInt(value)**
 - Integer number appended with an n (Eg: 5n)

Data Types

- **Undefined**
- **Null**
- **Symbol: Unique identifiers (e.g., `Symbol('id')`)**

`let age;
console.log(age);`

`let student = null;
console.log(student);`

`let id1 = Symbol("id");
let id2 = Symbol("id");`

`console.log(id1 === id2);`

Non-Primitive

- **Object: e.g., `{ name: "Alice" }`**

`let student = {
 name: "Alice",
 age: 20,
 city: "Chennai"
};`

`console.log(student.name);`

Variables

- **Syntax:**
 - **var identifier_name;**
 - **identifier_name;**
- **Best Practice:**
 1. **Give meaningful names to identifiers.**
 2. **Identifiers should not be too long or too short (min 3 character length)**
- **typedef – keyword**
 - **Returns the data type of a variable**
 - **Syntax: typedef identifier_name;**

Variables

- **typeof – Keyword**
 - Returns the data type of a variable
 - Syntax: `typeof identifier_name;`
 - Possible types – number, string, boolean, object, undefined, BigInt.
- **let – Keyword**
 - Declare variables
 - Syntax: `let identifier_name;`
 - Identifier must be declared before usage.
- **const – Keyword**
 - Syntax: `const var/let identifier_name = value;`
 - Values once assigned cannot be changed.

`let age = 25;`
`console.log(typeof age);`
Output: number

`let name = "Alice";`
`let age = 20;`

`const PI = 3.14;`
`// PI = 3.14159; // Error`

Difference – var, let, const

	Scope	Redeclare	Reassign	Hoisted	Binds this
var	No	Yes	Yes	Yes	Yes
let	Yes	No	Yes	No	No
const	Yes	No	No	No	No

Type Conversion

- **Implicit Type Conversion**  `console.log("10" + 5);`
- **Explicit Type Conversion**
 - **Number()** 

<code>Number("123")</code>	<code>Number("Hello")</code>
Output: 123	Output: NaN
 - **undefined, nan, non-numeric string => Nan**
 - **BigInt()**
 - **parseInt()**
 - **parseFloat()**
 - **String()**
 - **.toString()**
 - **Boolean()**

Control Structures

- **Same as C++ and Java**
 - **Selection/Branching Statements**
 - **Looping Statements**
 - **while**
 - **do .. while**
 - **for**
 - **for .. in**
 - **for .. of**
- **Best Practice: Add comments on the outermost nested control structure**

Control Structures

- **Same as C++ and Java**
 - **Selection/Branching Statements**
 - **Looping Statements**
- **Best Practice: Add comments on the outermost nested control structure**
- **0, NaN, "" (the empty string), null and undefined – Evaluates to false.**
- **Other values, [], {} – Evaluates to true.**

Arrays

- **Javascript-Specific Features**

- **Array Methods**

- **array_identifier_name.push(value);**
- **array_identifier_name.pop();**
- **array_identifier_name.shift(); //Dequeue**
- **array_identifier_name.unshift(value); //Enqueue**
- **array_identifier_name.sort();**
- **array_identifier_name.reverse();**
- **array_identifier_name.join(); – joins contents into a string**
- **array_identifier_name.concat(value/array);**



```
let fruits = ["Apple", "Banana", "Mango"];  
fruits.shift();  
console.log(fruits);  
Output: ["Banana", "Mango"]
```

Functions

- Same as C++ and Java
- **Function Declaration and Definition Syntax:**

function identifier_name (parameters)

{

Function block

optional return statement;

}

```
function square(x) {  
    return x * x;  
}
```

```
console.log(square(5));
```

- **Function Call/ Invocation Syntax:**

identifier_name(parameters);

```
greet();
```

```
function greet() {  
    console.log("Welcome!");  
}
```

- **Can be hoisted**

```
let add = function(a, b) {  
    return a + b;  
};
```

- **Function Literal**

```
console.log(add(10, 20));
```

Functions

- **Call - less parameters**
 - **Left out parameters – undefined**
- **Call - more parameters**
 - **nameless**
 - **arguments array**
- **Recursive Functions**
- **Function Overloading**

Functions

- **Call - less parameters**
 - **Left out parameters – undefined**
- **Call - more parameters**
 - **nameless**
 - **arguments array**
- **Optional Parameters**
- **Recursive Functions**
- **Function Overloading**

Input - Prompt()

- **Syntax:**
 - **prompt(“text to display”, “default text response);**
 - **returns text**
 - **Empty prompt returns null.**

Assignment - 1

- **Write a Javascript program to check whether an entered number is an Armstrong number or not.**

Best Practices

- 1. Use HTML tags with the same case (preferably lowercase).**
- 2. Filename to be based on functionality.**
- 3. When opening a tag, have a "tab space" for each line until you close the tag. It helps in the readability of your HTML document.**
- 4. Give a comment describing a web page as the first line of your HTML document.**
- 5. Keep HTML, CSS, and Javascript files separate.**
- 6. Give meaningful names to identifiers.**
- 7. Identifiers should not be too long or too short (min 3 character length).**
- 8. Add comments on the outermost nested control structure.**

1. **Paul J Deitel, Harvey M Deitel, and Abbey Deitel, "Internet and the world wide web: How to program," Pearson, Fifth Edition, 2012.**

THANK YOU

SHIV NADAR
— UNIVERSITY —
CHENNAI